

VISUAL FIELD GUIDE · LINKEDIN EDITION

AI Systems: সহজ বাংলায়

Context Engineering, RAG, MCP, Agents,
Harness, Loop - একটি Visual Field Guide

Software Engineers-এর জন্য একটি Practical Handbook

Shahriar Rashid

লেখক ও সংকলক · linkedin.com/in/shahriarrashid67

VERSION 1.0 · SEPTEMBER 2026

এই HANDBOOK কেন

Termগুলো কোথায় কাজে লাগে - সেটাই আগে বুঝে নেওয়া যাক

কোনো AI feature নিয়ে কাজ শুরু করলে Prompt, Context, RAG, Agent, Harness - সব শব্দই সামনে আসে। আলাদা আলাদা definition জানার চেয়ে system-এর কোন অংশে কোনটি কাজ করে, সেই সম্পর্কটা বোঝাই বেশি useful।

01

আলোচনার context ধরতে

Team discussion, design review বা vendor pitch-এ একটি term কোন অর্থে ব্যবহার হচ্ছে, তা বুঝতে এই map কাজে দেবে।

02

পুরো systemটা দেখতে

Model-এর পাশাপাশি context, retrieval, tools, loop, eval এবং safety - কোন অংশটি কোথায় বসে, সেই picture পরিষ্কার হবে।

লেখকের কথা

এই handbook textbook হওয়ার চেষ্টা করছে না। নতুন কোনো AI term শুনলে যেন দ্রুত বুঝতে পারেন - এটি model-এর বিষয়, context-এর বিষয়, নাকি system design-এর বিষয়। প্রতিষ্ঠিত English term রেখেছি; ব্যাখ্যা করেছি সহজ বাংলায়।

গণিত বা Machine Learning-এর গভীর জ্ঞান দরকার নেই। Programming, API এবং সাধারণ system design সম্পর্কে ধারণা থাকলেই যথেষ্ট।

MAP, NOT A CHECKLIST

১৪টি মূল অধ্যায় • ৩টি Part

Part I - ভিত্তি

1. AI থেকে LLM
2. Transformer ও Attention
3. Token, Embedding ও Training
4. Inference ও Context Window
5. Prompt Engineering
6. Context Engineering

Part II - AI সিস্টেম তৈরি

1. RAG
2. Tool / Function Calling ও MCP
3. Agent ও Agentic Loop
4. Sub-agent ও Multi-agent

Part III - Production

1. Harness Engineering
2. Loop Engineering
3. Evaluation
4. Production: Cost, Performance ও Risk

শেষের অংশ

1. নতুন ও evolving terms
2. ৩০ দিনের শেখার পথ
3. Production checklist
4. দ্রুত অভিধান
5. উৎস ও ক্রেডিট

সময় কম হলে

Context Window → Context Engineering → RAG → Agentic Loop → Harness → Evals - আগে এই ছয়টি topic পড়ুন। Modern AI application-এর মূল architecture ধরতে সুবিধা হবে।

কীভাবে পড়বেন

প্রথমবার ধারাবাহিকভাবে পড়তে পারেন। পরে glossary ও chapter opener-গুলো প্রয়োজনমতো reference হিসেবে ব্যবহার করুন।

PART I

ভিত্তি

Model কী, input কীভাবে ভাঙে, output কীভাবে তৈরি হয়, আর context কেন ফলাফলের মান বদলে দেয় - এই Part সেই foundation তৈরি করবে।

AI → LLM

Transformer

Token

Inference

Prompt

Context

● অধ্যায় ১

AI থেকে LLM: একই জিনিস নয়

কেন গুরুত্বপূর্ণ: “AI” শব্দটি অনেক বড় umbrella। কোন স্তর নিয়ে কথা হচ্ছে না বুঝলে architecture আর capability - দুটোই ভুল বোঝা যায়।

Artificial Intelligence - বুদ্ধিমত্তার মতো কাজ করানো

Machine Learning - data থেকে pattern শেখা

Deep Learning - বহু-layer neural network

Generative AI - নতুন content তৈরি

LLM - language token predict করা

চিত্র ১ · ধারণাগুলো সাধারণত nested; প্রতিটি AI system LLM নয়।

LLM - practical definition

বড় text corpus থেকে pattern শেখা একটি language model, যা আগের token-গুলোর ভিত্তিতে পরের token-এর probability হিসাব করে। Chat, code, summarization - এসব capability এই prediction process-এর উপর গড়ে ওঠে।

মনে রাখুন

Model fluent হলেই তার প্রতিটি বক্তব্য সত্য হয় না। Fluency হলো language modelling-এর ফল; factual reliability আলাদা engineering problem।

● অধ্যায় ২

Transformer ও Attention

কেন গুরুত্বপূর্ণ: LLM-এর ভেতরের সব mathematics না জানলেও Attention কী সমস্যার সমাধান করে - এটা জানা দরকার।

ছোট উদাহরণ

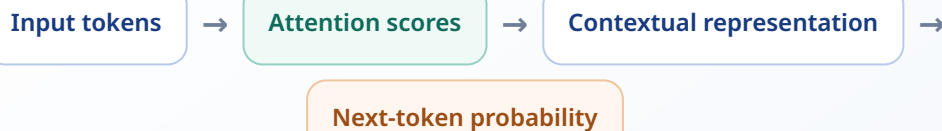
“রাকিব ব্যাংকে গেল, কারণ সে টাকা তুলবে।” এখানে “সে” কোন ব্যক্তিকে বোঝাচ্ছে? বাক্যের অন্য token-গুলোর সঙ্গে সম্পর্ক বিবেচনা না করলে উত্তর দেওয়া কঠিন।

Transformer

২০১৭ সালের Attention Is All You Need paper-এ প্রস্তাবিত architecture। Recurrent step-এর বদলে attention-ভিত্তিক computation ব্যবহার করে sequence modelling-কে বেশি parallelizable করেছে।

Attention

একটি token বোঝা বা পরের token predict করার সময় input-এর কোন অংশ কতটা প্রাসঙ্গিক - model সেই সম্পর্কের weight হিসাব করে।

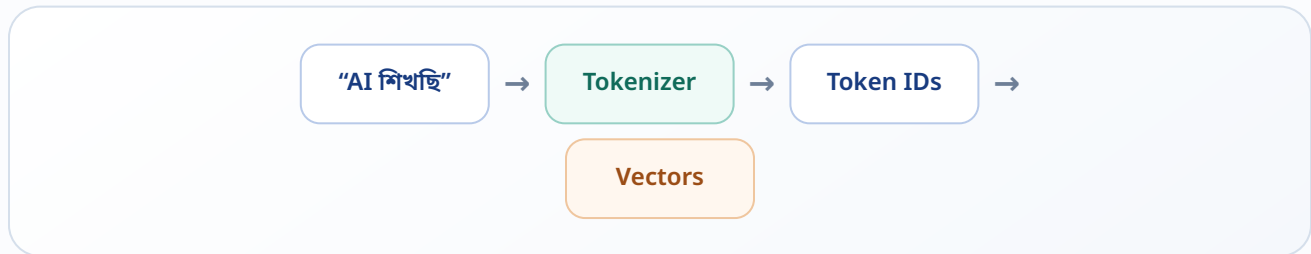
**Common mistake**

“Attention মানে model মানুষের মতো মনোযোগ দেয়” - এটা শুধু analogy। বাস্তবে এটি vector ও weight-এর computation।

● অধ্যায় ৩

Token, Tokenizer ও Embedding

কেন গুরুত্বপূর্ণ: Context limit, latency এবং cost - সবকিছুর হিসাব শেষ পর্যন্ত token-এ এসে দাঁড়ায়। Retrieval আবার embedding-এর উপর নির্ভর করে।



Token

Model-এর processing unit।
এটি পুরো শব্দ, শব্দের অংশ,
punctuation বা অন্য কোনো text
fragment হতে পারে।

Tokenizer

Text-কে token-এ ভেঙে প্রতিটির
numeric ID তৈরি করে। Token
count model ও tokenizer ভেদে
বদলায়।

Embedding

Token, বাক্য বা document-কে
vector-এ রূপ দেয়, যাতে
semantic similarity
গাণিতিকভাবে তুলনা করা যায়।

বাংলা text-এর practical note

একই অর্থের English ও Bangla text সব tokenizer-এ সমান token নেয় না। তাই “এক পৃষ্ঠা = নির্দিষ্ট token” ধরে budget করবেন না; ব্যবহৃত model-এর tokenizer দিয়ে মাপুন।

Engineering connection: Embedding search সাধারণত “একই শব্দ আছে কি?” নয়; “অর্থো কাছাকাছি কি?” - এই প্রশ্নের উত্তর খোঁজে। তবে domain vocabulary-এর জন্য keyword search এখনো useful হতে পারে।

MODEL LIFECYCLE

Parameter থেকে Fine-tuning

Term	সহজ ব্যাখ্যা	Engineering implication
Parameter / Weight	Training-এর সময় শেখা numeric value	বড় parameter count মানেই সব task-এ ভালো নয়
Pretraining	বড় corpus থেকে সাধারণ pattern শেখা	ব্যয়বহুল; সাধারণ app team এটি করে না
Fine-tuning	নির্দিষ্ট behaviour বা task-এর জন্য অতিরিক্ত training	Knowledge refresh-এর একমাত্র উপায় নয়
RLHF / RLAIF	মানুষ বা AI feedback থেকে preferred behaviour শেখানো	Helpful response style ও safety-তে ভূমিকা রাখে
Distillation	বড় model-এর behaviour থেকে ছোট model শেখানো	Cost/latency কমানোর পথ হতে পারে
Quantization	Weight কম precision-এ রাখা	Memory কমে; quality trade-off হতে পারে

Open-weight

Weights download করে নিজে চালানো যায়। License, training data এবং source code আলাদাভাবে open নাও হতে পারে।

Closed / hosted

সাধারণত API বা hosted product দিয়ে ব্যবহার করা হয়; infrastructure provider পরিচালনা করে।

Decision rule

Knowledge বদলাতে হলে আগে RAG বা context দেখুন। Consistent style/behaviour শেখাতে fine-tuning বিবেচনা করুন।

● অধ্যায় ৪

Inference: model যখন উত্তর দেয়

কেন গুরুত্বপূর্ণ: Training ও inference আলাদা phase। Production latency, throughput এবং usage cost মূলত inference design-এর সঙ্গে জড়িত।

Inference

Trained model-এ input পাঠিয়ে output তৈরি করার process। প্রতিটি request-এ model এক বা একাধিক token generate করে এবং সেই output আবার পরের ধাপের context হয়।

Temperature

Sampling কতটা varied হবে তার control। “0 মানেই mathematically deterministic” - এমন নিশ্চয়তা সব provider দেয় না।

Top-p / Top-k

পরের token কোন probability set থেকে sample হবে তা সীমিত করে। সাধারণত একসঙ্গে সব knob নাড়ানো দরকার হয় না।

Max output

Response length-এর upper bound। এটি context budget-এর একটি অংশ এবং cost/latency-তেও প্রভাব ফেলে।

Streaming

পুরো response শেষ হওয়ার আগে token পাঠানো। Perceived latency কমে; total generation time একইও থাকতে পারে।

TTFT · প্রথম token আসতে সময়

TPS · প্রতি সেকেন্ডে token

Latency · মোট request time

Throughput · unit time-এ কাজ

THE WORKING MEMORY

Context Window

কেন গুরুত্বপূর্ণ: Model একবারে কত input ও generated output বিবেচনা করতে পারে - এই সীমা system design বদলে দেয়।

System / developer instructions

Tool definitions ও schemas

Retrieved docs, files, memory

Conversation ও tool results

Current request + output-এর জায়গা

RAM analogy - সীমাসহ

Context window-কে working memory ভাবা যায়, কিন্তু এটি computer RAM-এর মতো exact mechanism নয়। পুরোনো message “মনে রাখা” সাধারণত history আবার পাঠানো, summary করা, অথবা external memory থেকে আনার ফল।

- ! Limit বড় হলেই সব তথ্য সমানভাবে ব্যবহার হবে - এমন নয়।
- ! অপ্রাসঙ্গিক context attention dilute করতে পারে।
- ! Input-এর সঙ্গে output budget-ও রাখতে হয়।

● অধ্যায় ৫

Prompt Engineering

কেন গুরুত্বপূর্ণ: অস্পষ্ট request থেকে model-কে লক্ষ্য, constraint এবং expected output বুঝতে হয়। Prompt সেই contract-এর প্রথম খসড়া।

Role → আপনি senior backend reviewer
Goal → এই API design-এর ব্লকি খুঁজুন
Context → Node.js, PostgreSQL, 5k req/min
Constraints → schema বদলানো যাবে না
Output → severity অনুযায়ী table
Examples → একটি good / bad finding

ভালো prompt কী করে

Ambiguity কমায়, success criteria জানায়,
প্রয়োজনীয় context দেয় এবং output parse করা
সহজ করে।

ভালো prompt কী করতে পারে না

Missing data বানিয়ে দিতে পারে না, permission
তৈরি করতে পারে না, আর weak system design
পুরোপুরি ঢাকতে পারে না।

Common mistake

শুধু prompt বড় করা। অতিরিক্ত instruction conflict তৈরি করতে পারে। আগে task,
evidence এবং output contract পরিষ্কার করুন।

PATTERNS

Prompt-এর reusable pattern

Zero-shot

কোনো example ছাড়া direct instruction। Simple, well-known task-এর জন্য ভালো starting point।

Few-shot

কয়েকটি input-output example দিয়ে pattern দেখানো। Format ও edge case শেখাতে useful।

Structured Output

JSON schema বা নির্দিষ্ট structure চাওয়া।
Downstream code-এ parsing ও validation সহজ হয়।

Decomposition

একটি বড় task-কে explicit ধাপে ভাগ করা। প্রতিটি ধাপে verification রাখা সহজ হয়।

BACKEND SCENARIO

“Bug fix করুন” বলার বদলে reproduction steps, relevant files, test command, allowed scope এবং acceptance criteria দিন। এতে model-এর guess কমে এবং review করা সহজ হয়।

বাস্তব rule

Prompt version করুন, কিন্তু prompt-কে একা optimize করবেন না। একইসঙ্গে retrieved context, tool result এবং eval score দেখুন।

Chain-of-thought নিয়ে সতর্কতা

Model-কে private reasoning প্রকাশ করতে বাধ্য করার বদলে concise rationale, assumptions, evidence বা verification steps চাইুন।

● অধ্যায় ৬

Context Engineering

কেন গুরুত্বপূর্ণ: ভালো model ও ভালো prompt থাকলেও ভুল file, stale history বা irrelevant tool result দিলে output খারাপ হতে পারে।

একটি পরিচিত ENGINEERING SCENARIO

Agent-কে bug fix করতে বলা হলো। Prompt পরিষ্কার, model শক্তিশালী। কিন্তু context-এ outdated API spec আছে, test failure নেই, আর relevant service file বাদ পড়েছে। Model সুন্দর code লিখল - ভুল problem-এর জন্য।

Context Engineering হলো model-কে “আরও বেশি” তথ্য দেওয়া নয়; সঠিক সময়ে সঠিক তথ্য দেওয়া।

Select

কোন instruction, file, memory বা evidence দরকার?

Order

কোন তথ্য আগে, কোনটি পাশে, কোনটি summary হবে?

Refresh

Stale context কখন বাদ যাবে বা নতুন করে retrieve হবে?

Context rot

Context বড় হতে হতে relevant signal হারিয়ে গেলে বা conflicting history জমলে quality কমতে পারে। এটি fixed threshold নয়; task ও model ভেদে বদলায়।

CONTEXT PIPELINE

Context কীভাবে গঠন করবেন

Task বোঝা

→

Source বাছাই

→

Rank / trim

→

Pack + verify

Instruction hierarchy

Stable policy, project rule এবং current request আলাদা layer-এ রাখুন; conflict হলে precedence পরিষ্কার করুন।

Retrieval

Query অনুযায়ী relevant doc বা code আনুন। “সব file পাঠানো” সাধারণত সবচেয়ে ভালো strategy নয়।

Memory

Long-term preference বা state external store-এ রাখুন; দরকারে retrieve করুন। Memory মানেই context window নয়।

Compaction

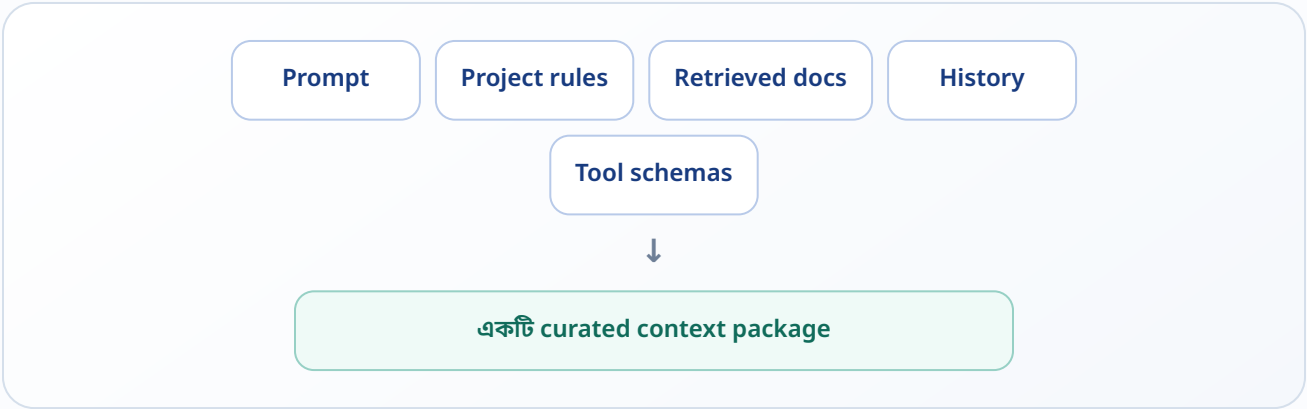
পুরোনো history summarize বা trim করুন। Summary-তে সিদ্ধান্ত, open issue ও evidence source রাখুন।

- ✓ Relevant source-এর date/version আছে?
- ✓ Duplicate বা conflicting instruction আছে?
- ✓ Tool output-এর raw noise trim করা হয়েছে?
- ✓ Answer-এর জন্য জায়গা রাখা হয়েছে?

COMPARISON

Prompt বনাম Context Engineering

প্রশ্ন	Prompt Engineering	Context Engineering
মূল focus	Instruction কীভাবে লেখা	Model কী কী information দেখছে
Typical input	Role, goal, constraint, format	Files, history, memory, retrieved docs, tool schema
Common failure	Ambiguous task বা output	Missing, stale বা noisy evidence
Optimization	Wording ও examples	Selection, ranking, ordering, trimming
Measure	Instruction-following success	Evidence coverage ও task success



Practical takeaway

Prompt একটি গুরুত্বপূর্ণ input; Context Engineering পুরো input environment-এর design। Production agent-এ দ্বিতীয়টির scope বড়।



PART II

AI সিস্টেম তৈরি

Model-কে নিজের data দেখানো, external action করানো এবং
একাধিক ধাপ চালানো - এখানেই chatbot থেকে application-এর যাত্রা
শুরু।

RAG

Tools

MCP

Agent

Loop

Sub-agent

● অধ্যায় ৭

RAG: নিজের data থেকে উত্তর

কেন গুরুত্বপূর্ণ: Model-এর training knowledge স্থির। কিন্তু product docs, policy বা customer data বদলায়। RAG request-এর সময় relevant evidence এনে দেয়।

Retrieval-Augmented Generation

User query অনুযায়ী external source থেকে relevant passage খুঁজে এনে prompt/context-এ যোগ করা, তারপর সেই evidence-এর ভিত্তিতে answer generate করা।

Documents

→

Chunk + index

→

Retrieve

→

Generate + cite

বাস্তব উদাহরণ

Internal leave-policy assistant। Policy PDF update হলে model retrain না করে index refresh করা যায়; answer-এর সঙ্গে source section দেখানোও সম্ভব।

RAG কী নয়

Vector database যোগ করলেই trustworthy answer নিশ্চিত হয় না। Retrieval ভুল হলে generation-ও ভুল evidence-এর উপর দাঁড়াবে।

RETRIEVAL PIPELINE

RAG-এর কাজের ধাপ

1

Ingest

PDF, web page, database row বা code থেকে clean text ও metadata তৈরি।

2

Chunk

Meaningful unit-এ ভাগ। খুব ছোট হলে context হারায়; খুব বড় হলে retrieval blunt হয়।

3

Index

Embedding/vector, keyword index বা দুটোই তৈরি। Metadata filter রাখুন।

4

Retrieve + rerank

Candidate বের করে relevance অনুযায়ী সাজান; duplicate ও stale result বাদ দিন।

5

Generate

Evidence context-এ দিন; unsupported claim না করতে স্পষ্ট নির্দেশ দিন।

6

Evaluate

Retrieval recall এবং answer correctness আলাদাভাবে মাপুন।

Hybrid search

Semantic embedding search-এর সঙ্গে keyword/BM25 search মিলিয়ে domain term, ID, error code বা exact phrase ধরার সম্ভাবনা বাড়ানো যায়।

CHOOSING THE APPROACH

RAG, Long Context, নাকি Fine-tuning?

আপনার প্রয়োজন	ভালো starting point	কারণ
Data প্রায়ই বদলায়	RAG	Index update করা যায়; retraining লাগে না
Corpus ছোট, request কম	Long context	Pipeline সহজ; retrieval failure নেই
Style/format consistent করা	Fine-tuning	Behaviour শেখানো যায়
Source citation দরকার	RAG	Retrieved evidence track করা যায়
সবকিছু local রাখতে হবে	Self-hosted model + retrieval	Privacy/control বাড়ে; ops cost-ও বাড়ে

Retrieval failure
সঠিক document পাওয়া যায়নি।
Generation যত ভালোই হোক, evidence missing।

Generation failure
সঠিক evidence ছিল, কিন্তু model ভুলভাবে ব্যবহার করেছে বা citation mismatch করেছে।

Debugging rule: “RAG খারাপ” বলার আগে retrieval result আলাদা করে দেখুন। Query, filter, chunk, rank এবং final context - প্রতিটি ধাপ trace করুন।

● অধ্যায় ৮

Tool Use ও Function Calling

কেন গুরুত্বপূর্ণ: Model text generate করে; real action আপনার code করে। এই boundary পরিষ্কার না হলে permission ও reliability - দুটোতেই সমস্যা হয়।

User request

→

Model selects tool

→

Your code executes

→

Result to model

Tool schema

Tool-এর নাম, purpose, input field, type ও constraint। Schema যত পরিষ্কার, ভুল call তত কম।

Validation

Model-generated argument-কে untrusted input ভাবুন। Type, authorization এবং business rule validate করুন।

Idempotency

Retry হলে double payment বা duplicate ticket যেন না হয় - mutation tool-এ key রাখুন।

Human approval

High-impact action - delete, send, publish, purchase - আগে confirmation বা policy gate দিন।

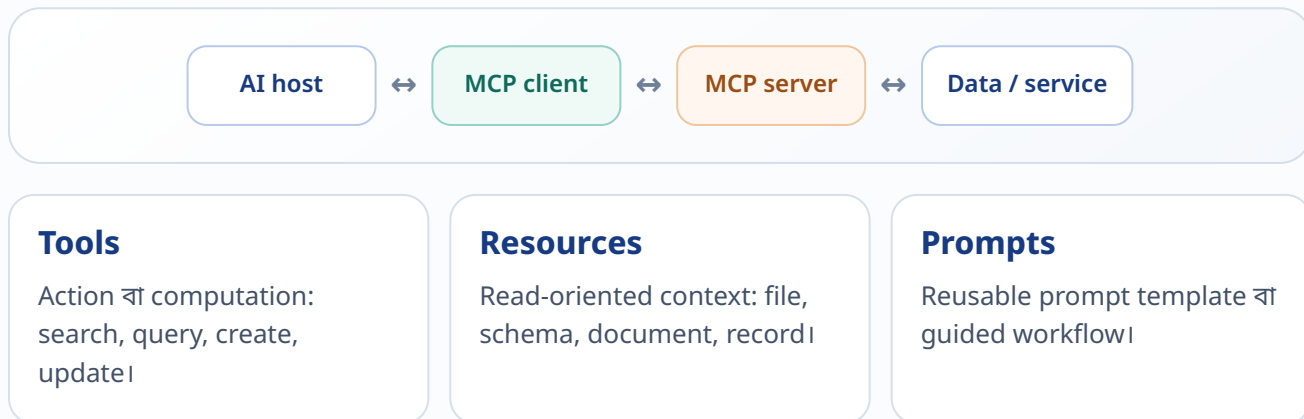
মনে রাখুন

Model tool “চালায়” না; model একটি structured request প্রস্তাব করে। Application সিদ্ধান্ত নেয় সেটি চালানো নিরাপদ ও অনুমোদিত কি না।

OPEN PROTOCOL

MCP: integration-এর common layer

Model Context Protocol (MCP) হলো AI application-কে external tools, resources ও prompts-এর সঙ্গে যুক্ত করার একটি open protocol।



কেন useful

প্রতিটি AI host-এর জন্য আলাদা integration layer না লিখে একটি standardized interface ব্যবহার করা যায়। তবে authentication, authorization এবং deployment এখনও আপনার design concern।

“AI-এর USB-C”

এটি জনপ্রিয় analogy, formal definition নয়। MCP data connector-এর চেয়ে বেশি; protocol lifecycle, capabilities ও transports-ও নির্ধারণ করে।

● অধ্যায় ৯

Agent: model + tools + সিদ্ধান্ত

কেন গুরুত্বপূর্ণ: Fixed workflow আর agent এক নয়। Agent intermediate result দেখে পরের action বেছে নিতে পারে।

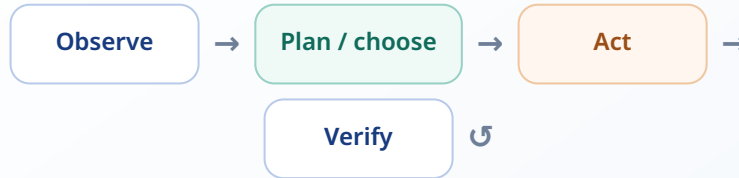
	Workflow	Agent
Path	আগে থেকে নির্ধারিত	Run-time-এ model বেছে নেয়
Predictability	সাধারণত বেশি	সাধারণত কম
Flexibility	Known process-এর জন্য ভালো	Open-ended task-এ useful
Cost / risk	Control সহজ	Loop, tools ও retries-এর কারণে বাড়তে পারে

Agent - practical definition
এমন একটি system যেখানে model goal, available tools এবং current state দেখে এক বা একাধিক action বেছে নেয়; application সেই action execute করে এবং result আবার model-কে দেয়।

কখন AGENT দরকার নাও হতে পারে
যদি process সবসময় “form validate → database write → email” হয়, deterministic workflow যথেষ্ট। Agent তখন unnecessary variability যোগ করতে পারে।

THE INNER LOOP

Agentic Loop



State

Goal, progress, tool result এবং remaining budget - loop-এর current picture।

Stop condition

Success, failure, max step, cost limit বা human escalation - কখন loop থামবে।

Recovery

Tool error, invalid output বা timeout হলে retry, alternate path বা rollback।

Trace

কোন input-এ কোন tool call হলো এবং কী result ফিরল - debugging-এর record।

তিনটি সাধারণ failure

Endless retry, একই ভুল plan পুনরাবৃত্তি, আর “কাজ হয়েছে” ধরে নেওয়া কিন্তু verification না করা।

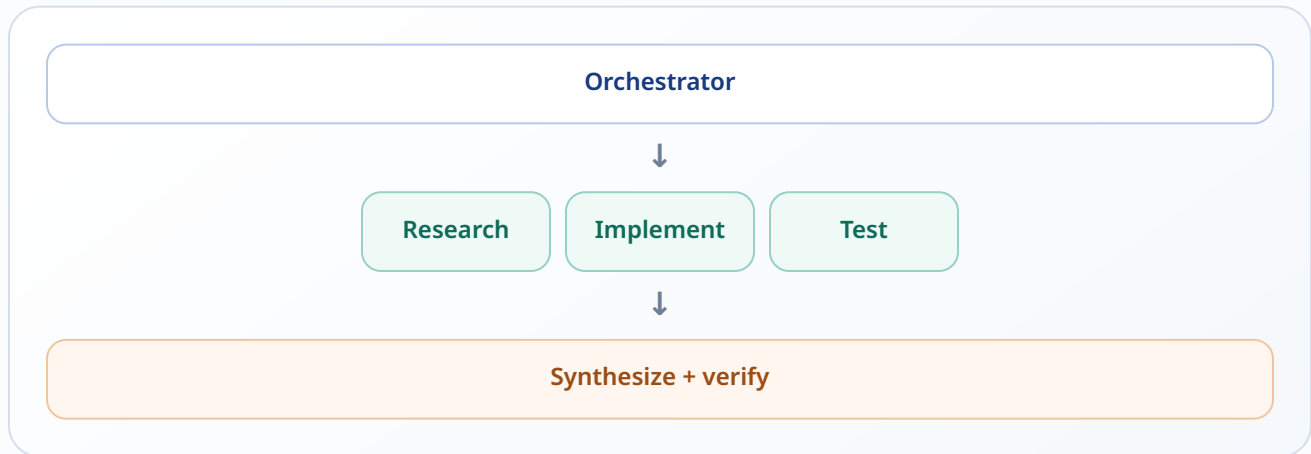
প্রথম design question

Agent কী করতে পারবে - এর আগে ঠিক করুন, কী করলে নিশ্চিতভাবে থামবে।

● অধ্যায় ১০

Sub-agent ও Multi-agent

কেন গুরুত্বপূর্ণ: বড় task ভাগ করলে context isolation ও parallelism পাওয়া যায়; কিন্তু coordination cost-ও তৈরি হয়।



Sub-agent

Parent agent-এর দেওয়া নির্দিষ্ট subtask আলাদা context-এ করে এবং concise result ফেরায়।

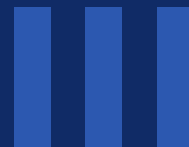
Orchestrator-worker

Central model run-time-এ subtasks নির্ধারণ, delegate এবং result merge করে।

- ✓ Subtask-এর boundary ও expected output স্পষ্ট করুন।
- ✓ একই file edit হলে ownership বা merge strategy দিন।
- ✓ Worker result blindly trust না করে integration test চালান।
- ✓ Parallelism-এর cost ও rate limit budget করুন।

ব্যবহার করবেন যখন

Subtasks সত্যিই independent, context আলাদা রাখা useful, অথবা specialist review দরকার। শুধু “আরও agent” মানেই দ্রুত বা ভালো নয়।



PART III

Production AI Engineering

Demo কাজ করলেই system production-ready হয় না। Reliability আসে model-এর চারপাশের control, evaluation, observability এবং safety থেকে।

Harness

Loop

Evals

Cost

Security

Safety

● অধ্যায় ১১

Harness Engineering

কেন গুরুত্বপূর্ণ: একই model ভিন্ন tools, context policy, runtime ও eval setup-এ খুব ভিন্ন ফল দিতে পারে।

Harness - practical definition

Model-কে ঘিরে থাকা software layer: instructions, context assembly, tool execution, permissions, memory, loop control, logging, eval এবং recovery।

Policies · permissions · budgets

Planner · loop · retries · routing

Context · memory · retrieval · tools

Model

Runtime · sandbox · observability

Term status

“Harness Engineering” একটি rapidly evolving practitioner term; এর scope সব team একইভাবে সংজ্ঞায়িত করে না। এই guide-এ এটি model-এর চারপাশের operational system বোঝাতে ব্যবহৃত হয়েছে।

HARNESS CHECKLIST

Harness-এর আটটি layer

1 · Instructions

Stable policy, task prompt, project conventions।

2 · Context

Retrieval, memory, compaction, file selection।

3 · Tools

Schema, validation, authentication, idempotency।

4 · Loop control

State, stop condition, retry, timeout, budget।

5 · Runtime

Sandbox, filesystem, network, dependency isolation।

6 · Observability

Trace, logs, token/cost, error classification।

7 · Evaluation

Golden set, regression, human review, grader।

8 · Governance

Access policy, PII, audit, retention, approval।

Debugging order

Model বদলানোর আগে দেখুন: model কি সঠিক context পেয়েছে, সঠিক tool call করতে পেরেছে, আর success যাচাই করেছে? অনেক failure harness-level।

● অধ্যায় ১২

Loop Engineering

কেন গুরুত্বপূর্ণ: Agentic loop-কে production process-এ চালাতে হলে শুধু “আবার চেষ্টা করো” যথেষ্ট নয়; feedback, verification ও stop policy design করতে হয়।

Term status

“Loop Engineering” এখনো standardized academic discipline নয়। Practitioner-রা autonomous বা semi-autonomous work cycle design, run এবং improve করার অর্থে শব্দটি ব্যবহার করছেন।

Goal / spec



Agent run



Tests / sensors



Feedback



Guides

কাজের আগে direction: spec, conventions, constraints, examples।

Sensors

কাজের পরে signal: tests, lint, screenshots, metrics, reviewer feedback।

CODING LOOP

Issue পড়া → code edit → unit test → failure inspect → targeted fix → full test → diff review → stop। প্রতিটি arrow-তে state ও evidence থাকে।

DESIGN FOR STOPPING

একটি নিরাপদ loop-এর anatomy

Component	Question	Example
Objective	কাজ শেষ মানে কী?	সব acceptance test pass
Feedback	ভুল কীভাবে জানা যাবে?	Test, typecheck, human review
Budget	কত step/time/cost?	Max 8 iterations, 15 min
Escalation	কখন মানুষকে ডাকবে?	Migration বা security change
Recovery	Partial failure হলে কী?	Checkpoint, retry, rollback
Evidence	Success-এর proof কী?	Logs + test artifact + diff

Endless loop
একই action বারবার। Step limit ও novelty check দিন।

Reward hacking
Metric pass, আসল goal fail। Multiple sensors ব্যবহার করুন।

Silent drift
সময় গেলে quality কমে। Regression eval চালান।

Production rule
Loop যত autonomous, verification তত independent হওয়া উচিত। একই model-এর self-check একমাত্র gate নয়।

● অধ্যায় ১৩

Evaluation: “কাজ করছে” মানে কী?

কেন গুরুত্বপূর্ণ: Demo success selection bias তৈরি করে। Evals ছাড়া prompt বা model change সত্যিই উন্নতি করেছে কি না বোঝা যায় না।

Golden set

Realistic, versioned test case যার expected outcome বা rubric জানা আছে।

Task metric

Exact match, pass rate, retrieval recall, citation support, latency বা cost।

Human review

Nuance, usefulness ও domain judgement - যা simple metric ধরতে পারে না।

LLM-as-a-judge

Scaled qualitative grading; rubric, calibration এবং bias check দরকার।

Baseline



Change one thing



Run eval



Inspect failures

Metric trap

Aggregate score ভালো হলেও critical slice খারাপ হতে পারে। Language, customer type, tool, document age এবং risk level অনুযায়ী failure slice দেখুন।

EVAL STACK

কী কী আলাদাভাবে মাপবেন

Layer	উদাহরণ metric	Failure বোঝায়
Retrieval	Recall@k, source freshness	Evidence পাওয়া যায়নি
Generation	Correctness, groundedness	Evidence ভুলভাবে ব্যবহার
Tool use	Schema validity, success rate	Wrong tool/argument/runtime
Agent	Task completion, steps, retries	Planning বা loop control
Product	Latency, cost, user resolution	System useful হলেও viable নয়
Safety	Policy violation, PII leakage	Guardrail বা permission gap

শুরু করার ছোট উপায়

Production log থেকে ৩০-৫০টি representative case নিন। Happy path-এর পাশাপাশি ambiguous, adversarial এবং tool-failure case রাখুন।

- ✓ Dataset versioned?
- ✓ Rubric স্পষ্ট?
- ✓ Baseline সংরক্ষিত?
- ✓ Failure sample manually reviewed?
- ✓ Cost ও latency score-এর পাশে আছে?

● অধ্যায় ১৪

Production: Cost, Performance ও Risk

কেন গুরুত্বপূর্ণ: Production system-এ quality শুধু model output নয়। Cost, latency, permissions এবং সম্ভাব্য ক্ষতির সীমা - সবই design-এর অংশ।

Model routing

Simple task ছোট/দ্রুত model-এ, কঠিন task বড় model-এ। Router-ও eval করতে হবে।

Prompt caching

Repeated prefix cache করলে supported provider-এ latency ও input cost কমতে পারে।

Batching

Immediate response দরকার নেই এমন workload queue/batch-এ পাঠানো যায়।

Context discipline

Duplicate history, raw logs ও irrelevant docs trim করলে cost ও noise - দুটোই কমে।

```
Total cost ≈ model tokens
+ tool / retrieval compute
+ retries and failed runs
+ storage, tracing and human review
```

Optimize the system

TPS বাড়ালেই user experience ভালো হবে না, যদি retrieval 4 সেকেন্ড নেয় বা agent পাঁচবার অপ্রয়োজনীয় tool call করে। End-to-end trace দেখুন।

SECURITY

Prompt Injection ও Tool Safety

Retrieved web page, email বা document-এর ভেতরের text instruction-এর মতো দেখাতে পারে। External content-কে trusted policy ভাবলে agent ভুল action নিতে পারে।

Prompt Injection

Untrusted content model-কে system goal থেকে সরাতে বা secret/action পেতে নির্দেশ দেয়।

Least Privilege

প্রতিটি tool-কে কাজের জন্য যতটুকু permission দরকার, শুধু ততটুকু দিন।

Sandbox

Code/file/network action isolated environment-এ চালান; resource limit দিন।

Approval gate

Send, delete, transfer, publish বা permission change-এর আগে human confirmation।

- ! Model output সরাসরি shell/SQL-এ execute করবেন না।
- ! Tool argument server-side validate করুন।
- ! Secret context-এ থাকলে model বা tool output দিয়ে leak হতে পারে।
- ! Audit log-এ sensitive data redaction দিন।

Boundary matters

Prompt দিয়ে security guarantee হয় না। Enforcement application code, identity system, sandbox ও policy layer-এ থাকতে হবে।

SAFETY & ALIGNMENT

একই শব্দ নয়: Security, Safety, Alignment

Term	মূল প্রশ্ন	উদাহরণ
Security	কে system misuse বা compromise করতে পারে?	Prompt injection, secret leak, unauthorized tool
Safety	System-এর action কী ক্ষতি করতে পারে?	Wrong medical advice, destructive automation
Alignment	Model-এর behaviour intended goal/ value-এর সঙ্গে মেলে?	Helpful, honest, policy-consistent response
Guardrail	কোন input/output/action আটকাব?	PII detector, allowlist, moderation
Red teaming	ইচ্ছা করে ভাঙলে কোথায় দুর্বল?	Adversarial prompt ও abuse scenario

HUMAN-IN-THE-LOOP

Human approval মানে সব request মানুষ দেখবে - এমন নয়। Risk-based escalation করুন: low-risk read auto, high-impact write review।

Sycophancy

Model ব্যবহারকারীর assumption-এর সঙ্গে অকারণে একমত হতে পারে। Review prompt-এ counter-evidence, uncertainty এবং alternative explanation চাইুন।

EVOLVING LANGUAGE

নতুন term - স্থির সংজ্ঞা নয়

AI engineering-এর vocabulary দ্রুত বদলায়। নিচের শব্দগুলো useful, তবে company বা লেখকভেদে scope আলাদা হতে পারে।

Context Rot

Long context-এ signal dilute, conflict বা stale history জমে quality কমে যাওয়ার informal description।

Compaction

দীর্ঘ session-এর history summarize/trim করে নতুন context space তৈরি করা।

Skills

দরকারে load হওয়া reusable instruction, script বা domain workflow bundle।

Computer Use

Screenshot/UI state দেখে mouse-keyboard-এর মতো interface দিয়ে কাজ করা।

Test-time Compute

Inference-এর সময় বেশি search/reasoning/sample খরচ করে answer improve করার family of techniques।

Vibe Coding

AI-generated code দ্রুত গ্রহণ করার informal practice; review ও ownership বাদ দিলে risk বাড়ে।

শব্দ যাচাইয়ের নিয়ম

কে শব্দটি ব্যবহার করছে, কোন system boundary বোঝাচ্ছে, এবং measurable behaviour কী - এই তিনটি প্রশ্ন করুন।

BUILD, DON'T ONLY READ

৩০ দিনের practical path

সপ্তাহ ১ • Prompt

একটি classification বা extraction task নিই। Zero-shot, few-shot ও structured output তুলনা করুন। ২০টি eval case রাখুন।

সপ্তাহ ২ • RAG + Tool

নিজের ছোট document set index করুন। Retrieval result trace করুন। একটি read-only tool যোগ করুন।

সপ্তাহ ৩ • Agent

Max step, timeout ও stop conditionসহ ছোট loop বানান। Failure log থেকে case সংগ্রহ করুন।

সপ্তাহ ৪ • Production

Eval, cost, latency, sandbox ও approval gate যোগ করুন। Model বদলে regression compare করুন।

একটি project idea

Repository issue assistant: issue পড়ে relevant file খুঁজবে, plan দেবে, patch প্রস্তাব করবে, test চালাবে - কিন্তু merge করবে না। এতে Context, RAG, Tools, Agent, Harness ও Evals একসঙ্গে শেখা যায়।

- ✓ প্রতিদিন ছোট deliverable
- ✓ প্রথম দিন থেকেই eval case
- ✓ প্রতিটি tool call trace
- ✓ Model change-এর আগে baseline
- ✓ Final action-এ human review

SHIP WITH EVIDENCE

Deploy-এর আগে ১৬টি প্রশ্ন

Task & Context

- ✓ Success criteria measurable?
- ✓ Source fresh ও versioned?
- ✓ Irrelevant context trim করা?
- ✓ Unknown হলে abstain path আছে?

Tools & Loop

- ✓ Arguments server-side validated?
- ✓ Mutation idempotent?
- ✓ Stop/timeout/budget আছে?
- ✓ Partial failure recoverable?

Evaluation

- ✓ Representative golden set?
- ✓ Critical slice আলাদা?
- ✓ Baseline ও regression tracked?
- ✓ Human review sample আছে?

Security & Ops

- ✓ Least privilege enforced?
- ✓ PII/secret handling defined?
- ✓ Trace, cost ও alert available?
- ✓ Rollback/kill switch tested?

শেষবার যা দেখবেন

Production-ready মানে system কখনো ভুল করবে না - এমন নয়। গুরুত্বপূর্ণ হলো failure mode জানা, সীমাবদ্ধতা স্পষ্ট করা, সম্ভাব্য ক্ষতি সীমিত রাখা এবং recovery আগে থেকে test করা।

QUICK REFERENCE

দ্রুত অভিধান · A-M

Agent

Goal ও state দেখে পরের action বেছে নেওয়া system।

Alignment

Intended goal ও value-এর সঙ্গে behaviour মেলানো।

Batching

একাধিক non-urgent request একসঙ্গে process করা।

Chunking

Document-কে retrieval-friendly unit-এ ভাগ করা।

Context Engineering

Relevant information select, order ও refresh করা।

Context Rot

দীর্ঘ/noisy context-এ signal দুর্বল হওয়ার informal term।

Embedding

Text/meaning-এর numeric vector representation।

Fine-tuning

নির্দিষ্ট behaviour/task-এর জন্য additional training।

Golden Set

Versioned evaluation case ও expected rubric।

Guardrail

Input, output বা action-এর enforced check।

Harness

Model-এর চারপাশের orchestration ও control layer।

Hybrid Search

Keyword ও semantic retrieval একসঙ্গে।

Latency / TTFT

মোট সময় / প্রথম token আসতে সময়।

LLM

Language token predict করতে trained model।

LoRA / PEFT

কম parameter update করে efficient fine-tuning।

Memory

Context-এর বাইরে রাখা persistent information।

Agentic Loop

Observe → act → verify পুনরাবৃত্ত cycle।

Attention

Input-এর কোন অংশ কতটা relevant তার computation।

Benchmark

Comparable, standardized test set বা task suite।

Compaction

পুরোনো context summarize বা trim করা।

Context Window

এক request-এ model যত token বিবেচনা করতে পারে।

Distillation

বড় model-এর behaviour থেকে ছোট model শেখানো।

Eval

নিজের task/data দিয়ে system quality মাপা।

Function Calling

Model-এর structured tool request mechanism।

Grounding

Answer-কে provided evidence-এর সঙ্গে বাঁধা।

Hallucination

Unsupported বা fabricated output।

HITL

Risky step-এ human review/approval।

Inference

Trained model দিয়ে output তৈরি করা।

Least Privilege

Minimum necessary permission।

LLM-as-a-Judge

এক model দিয়ে output grade করা।

MCP

AI app ও tools/resources-এর open protocol।

Model Routing

Task অনুযায়ী model নির্বাচন।

QUICK REFERENCE

দ্রুত অভিধান · N-Z

Open-weight

Weights download ও self-host করা যায় এমন model।

Parameter / Weight

Training-এ শেখা numeric value।

Prompt Engineering

Instruction, context ও output contract লেখা।

Quantization

কম precision-এ weight রেখে model ছোট করা।

Rate Limit

সময়ভিত্তিক request বা token সীমা।

RLHF / RLAI

Human বা AI feedback থেকে preference শেখানো।

Skills

Reusable instruction/workflow bundle।

Structured Output

Schema-constrained response।

Sycophancy

ব্যবহারকারীর সঙ্গে অকারণে একমত হওয়া।

Temperature

Sampling variation control।

Token

Model-এর text processing unit।

Tracing

Request, tool call, state ও result-এর record।

Vector Database

Vector similarity search-এর optimized store।

Workflow

আগে থেকে নির্ধারিত execution path।

Orchestrator-worker

Central agent task ভাগ ও result merge করে।

Prompt Caching

Repeated prompt prefix reuse করার optimization।

Prompt Injection

Untrusted content দিয়ে model-কে misdirect করা।

RAG

External evidence retrieve করে answer generate করা।

Reranker

Retrieved candidate-কে আবার relevance অনুযায়ী সাজায়।

Sandbox

Action চালানোর isolated, constrained environment।

Streaming

Generation চলাকালে token পাঠানো।

Sub-agent

আলাদা context-এ subtask করা agent।

System Prompt

Stable role, policy ও behaviour instruction।

Test-time Compute

Inference-এ বেশি compute দিয়ে answer improve করা।

Tool Schema

Tool name, purpose, parameter ও type definition।

Transformer

Attention-ভিত্তিক sequence architecture।

Vibe Coding

AI-assisted দ্রুত coding-এর informal term।

Zero-shot

Example ছাড়া direct instruction।

নোট

Glossary-এর definitionগুলো এই handbook-এর scope-এ concise করা হয়েছে; formal specification হিসেবে ব্যবহার করবেন না।

READ THE PRIMARY SOURCES

উৎস, সীমা ও AI disclosure

1. Vaswani et al., Attention Is All You Need (2017): arxiv.org/abs/1706.03762
2. Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020): arxiv.org/abs/2005.11401
3. Anthropic Engineering, Building effective agents: anthropic.com/engineering/building-effective-agents
4. Anthropic Engineering, Contextual Retrieval: anthropic.com/engineering/contextual-retrieval
5. Model Context Protocol documentation: modelcontextprotocol.io/docs
6. OpenAI, Evaluation best practices: developers.openai.com/api/docs/guides/evaluation-best-practices
7. OWASP GenAI Security Project: genai.owasp.org
8. Addy Osmani, practitioner writing on Loop Engineering: addyosmani.com/blog/loop-engineering

সম্পাদকীয় সীমা

Harness Engineering ও Loop Engineering-কে evolving practitioner terminology হিসেবে উপস্থাপন করা হয়েছে। Model limits, pricing এবং product behaviour বদলায় - deployment-এর আগে provider documentation দেখুন।

AI-assisted, human-curated

প্রাথমিক লেখা, structure, diagram ও Bangla drafting-এ Claude (Anthropic) সহায়তা করেছে। এই LinkedIn edition পুনর্গঠন, fact-checking ও visual editing-এ AI tools ব্যবহৃত হয়েছে; বিষয়বস্তু নির্বাচন, প্রকাশ ও দায় লেখক/সংকলকের।

শেষ কথা

Tool বদলাবে। Engineering-এর basics বদলাবে না।

সঠিক context দেওয়া, permission-এর boundary ঠিক রাখা, ফল মাপা এবং কাজটি সত্যিই হয়েছে কি না যাচাই করা - tool যেটাই হোক, এই অভ্যাসগুলো কাজে লাগে।

এই handbook আপনার কাজে লাগলে share করতে পারেন।
অনুগ্রহ করে author credit ও source note অক্ষত রাখুন।

Shahriar Rashid

linkedin.com/in/shahriarrashid67

Version 1.0 · September 2026